



REDUX TOOLKIT

Learning Documentation

Understanding Modern State Management
in React Applications



Mini Hackathon 2026

Technology Explored



React.js



Redux Toolkit



React Redux

Submitted By

Alok Kumar

Prepared as part of my Mini Hackathon Learning Journey

Table of Contents

S.no	Topic	Page Number
1	Introduction to Redux Toolkit	3
2	Why Redux Toolkit Is Used	4
3	Core Concepts	5-6
4	Data Flow	7-8
5	Folder Structure	9
6	Important Functions	10
7	My Learning	11
8	Real-World Use Cases	12
9	Challenges I Faced	13
10	Additional Things I Explored	13
11	Conclusion	14
12	References	14

Introduction to Redux Toolkit

Redux Toolkit (RTK) is the official and recommended way to use Redux in React applications. It helps the developers to manage application state in a simple, organized, and efficient way.

Before Redux Toolkit was introduced, using Redux required writing a lot of repetitive code, which makes it difficult for beginners to understand and maintain. Redux Toolkit solves this problem by providing built-in tools that reduce boilerplate code and make Redux easier to use.

With Redux Toolkit, developers can create the Redux store, manage state, and write application logic with much less code. It follows a structured approach, making applications easier to build, debug, and maintain. Today, Redux Toolkit is widely used in modern React projects because it improves both development speed and code quality.

◆ Key Features

- Official way to use Redux.
- Reduces boilerplate code.
- Easy to learn.
- Simple store setup.
- Better code organization.
- Scalable for large applications.

My Understanding

After exploring Redux Toolkit, I understood that it is not a replacement for React. Instead, it works with React to manage shared data more efficiently. It reduces the complexity of code and makes large applications easier to organize and maintain.

Why Is Redux Toolkit Used ?

Redux Toolkit is used to simplify Redux development. Before Redux Toolkit, developers used **Traditional Redux**, which required writing a lot of boilerplate code and manually configuring the Redux store. This made the development process more complex, especially for beginners.

◆ Key Reasons

- Reduces boilerplate code compared to **Traditional Redux**.
- Simplifies Redux store setup.
- Makes state management easier.
- Improves code readability.
- Easy for beginners to learn.
- Supports Redux DevTools.
- Follows official Redux best practices.
- Suitable for both small and large applications.

My Understanding

Redux Toolkit saves development time by reducing unnecessary code. It makes React applications easier to build and maintain.

Traditional Redux vs Redux Toolkit

✗ Traditional Redux	✓ Redux Toolkit
More boilerplate code	Less boilerplate code
Complex store setup	Simple store setup
More manual configuration	Built-in utilities
Harder for beginners	Beginner-friendly

Core Concepts

Redux Toolkit is based on a few core concepts that work together to manage the state of a React application. These concepts help developers organize data, update the application efficiently, and share information between different components.

◆ State

State is the data of an application. It stores information that can change while the application is running. Whenever the state changes, the user interface (UI) updates automatically.

Key Points

- Stores application data.
- Changes according to user actions.
- Used to update the UI.

Example

In an online shopping website, the following are examples of state:

- Shopping Cart
- User Login Status
- Dark/Light Theme
- Product List

◆ Store

The Store is the central place where Redux keeps the application's state. Instead of storing the same data in multiple components, Redux stores it in one location so every component can access it whenever required.

Key Points

- Stores the global state.
- Created using `configureStore()`.
- Makes data sharing easier.

Example

Instead of storing cart data separately in different components, Redux Toolkit stores it in one Store that every component can use.

◆ Action

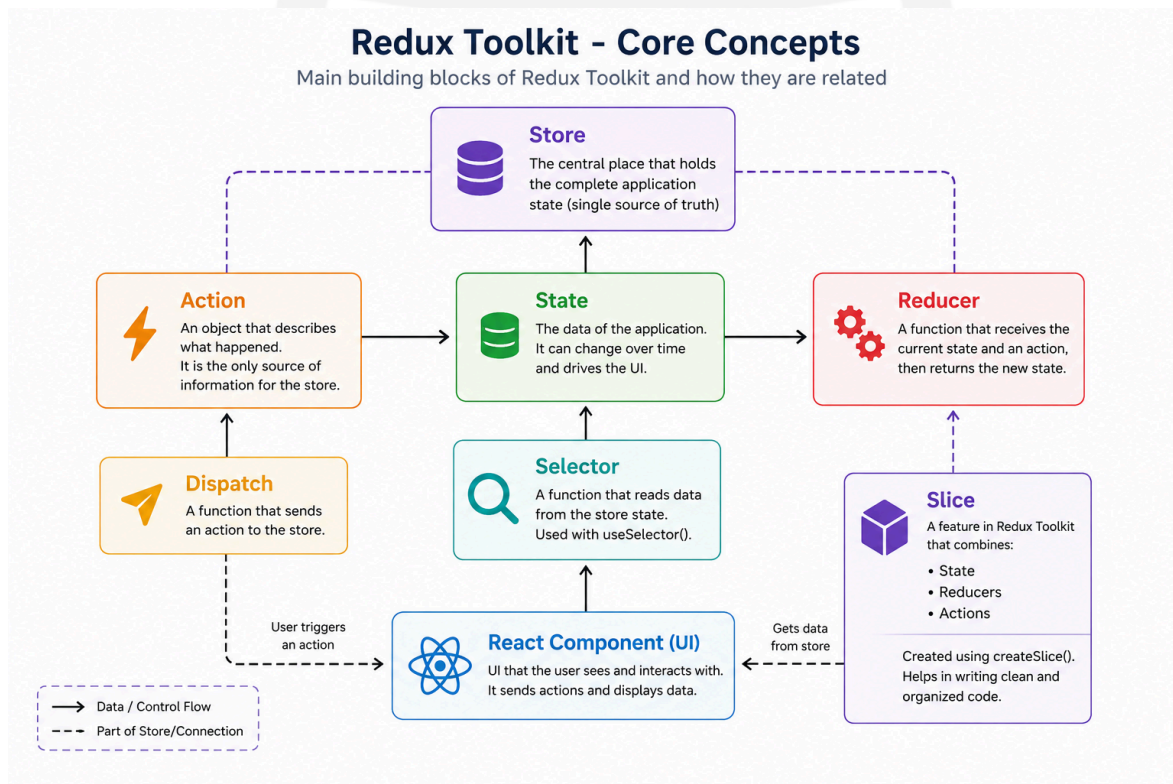
An Action is a message that tells Redux what needs to happen. It does not change the state directly but requests Redux to perform a specific update.

Key Points

- Represents a user event.
- Describes what should happen.
- Sent using `dispatch()`.

Example

When a user clicks the "Add to Cart" button, an action is dispatched to add that product to the shopping cart.



Data Flow

Redux Toolkit follows a simple and predictable data flow. Whenever a user performs an action, Redux updates the application state in a fixed sequence. This organized flow makes the application easier to understand, debug, and maintain.

Instead of changing the data directly, every update follows a step-by-step process. This ensures that the state remains consistent and all components receive the latest data.

◆ Data Flow Steps

1. User Action

The process starts when the user interacts with the application.

Examples:

- Clicking Add to Cart
- Logging in
- Removing an item

2. Dispatch

The React component sends an action to Redux using the `dispatch()` function.

Purpose

- Send a request to Redux.
- Starts the state update process.

3. Reducer

The reducer receives the current state and the action. It decides how the state should be updated and returns the new state.

Purpose

- Processes the action.
- Updates the application state.
- Returns the updated state.

4. Store

The updated state is saved in the Redux Store.

Purpose

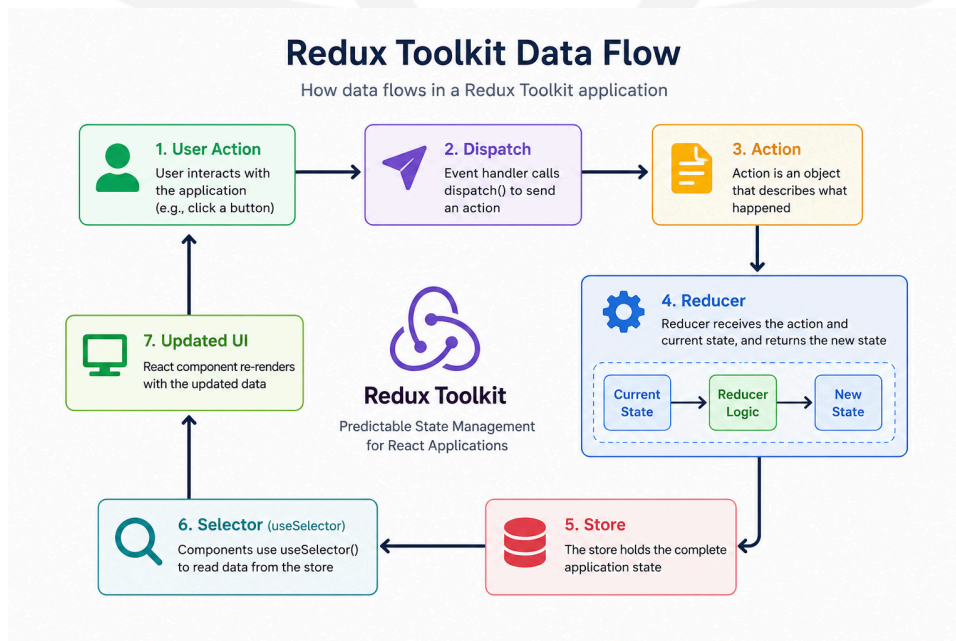
- Keeps the latest application data.
- Makes updated data available to all components.

5. Selector

React components use `useSelector()` to read the updated data.

Purpose

- Reads the required state.
- Displays the latest information.



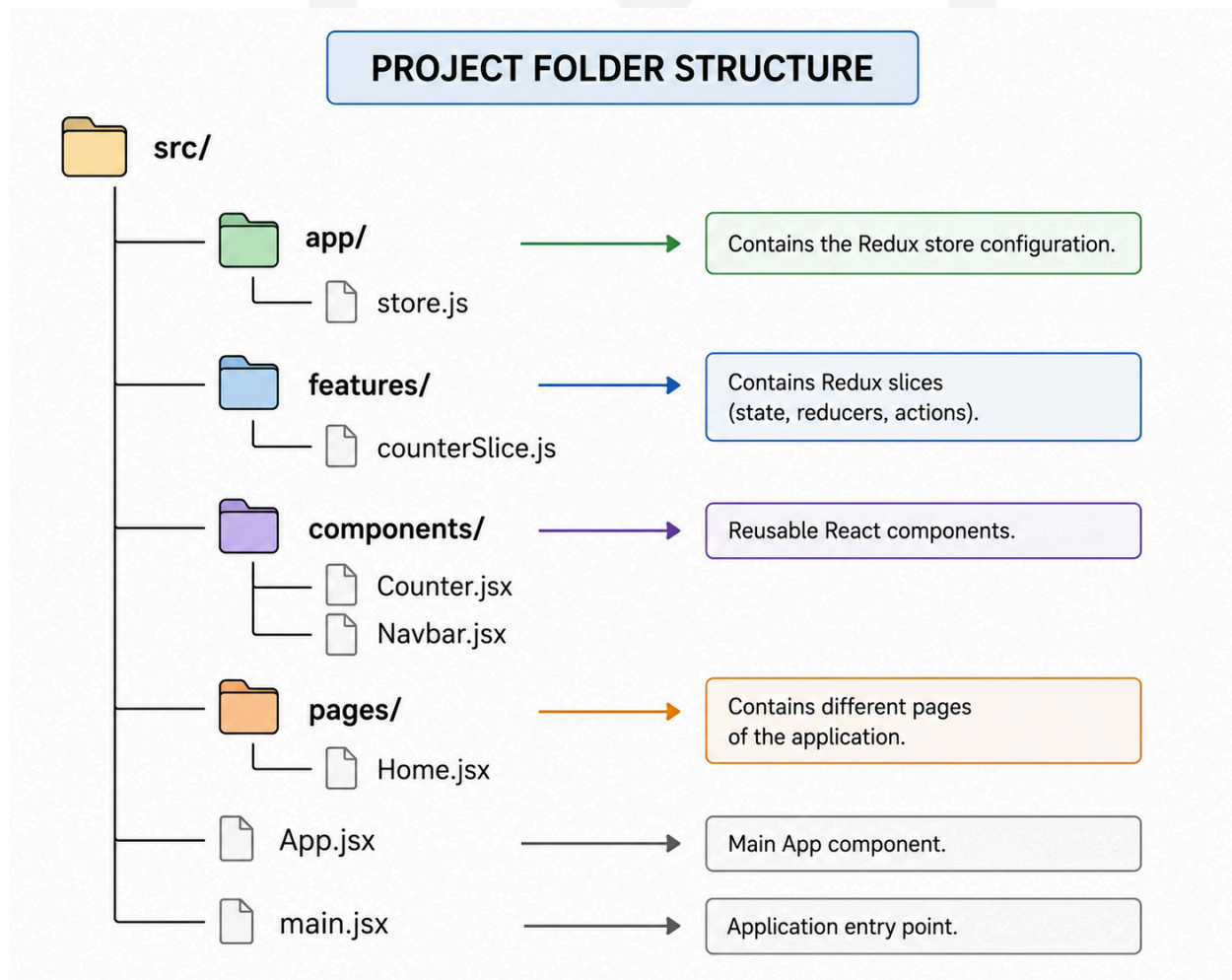
My Understanding

After learning Redux Toolkit Data Flow, I understood that every state update follows a fixed sequence instead of changing data directly.

Folder Structure

A well-organized folder structure makes a React application easier to understand and maintain. Redux Toolkit files are usually kept in separate folders so that each part of the project has a clear responsibility.

◆ Good Folder Structure



Folder Description

- **app/** → Contains Redux Store configuration.
- **features/** → Stores Redux slices.
- **components/** → Contains reusable React components.
- **pages/** → Stores application pages.
- **main.jsx** → Entry point of the application.

Important Functions

Redux Toolkit provides several built-in functions that simplify Redux development and reduce repetitive code.

configureStore()

Creates the Redux Store and automatically enables useful features.

Key Points

- Creates the Redux Store.
- Simplifies store configuration.

createSlice()

Creates a Redux Slice by combining state and reducers into a single file.

Key Points

- Reduces boilerplate code.
- Automatically generates actions.

createReducer()

Used to create reducers in a simpler and more readable way.

Key Points

- Simplifies reducer logic.
- Makes the code cleaner.

createAction()

Creates action creator functions without writing action objects manually.

Key Points

- Generates actions easily.
- Reduces repetitive code.

My Learning & Notes

During this mini hackathon, I learned that Redux Toolkit is a simpler and more organized way to manage state in React applications. It reduces repetitive code and makes Redux easier to understand, especially for beginners.

I also understood how the Store, State, Actions, Reducers, and Slices work together to manage application data. Learning the data flow and folder structure helped me understand how Redux Toolkit is used in real-world projects.

◆ My Key Learnings

- Redux Toolkit is easier than Traditional Redux.
- It reduces boilerplate code.
- State management becomes simple and organized.
- `createSlice()` combines state, reducers, and actions.
- `configureStore()` simplifies store setup.
- Predictable data flow makes debugging easier.

My Overall Understanding

After completing this documentation, I feel more confident using Redux Toolkit. It helped me understand how modern React applications manage shared data in a clean, efficient, and organized way.

Real World Use Cases

Redux Toolkit is used in many React applications where the same data needs to be used in different components. It makes state management simple and keeps the project organized.

E-Commerce Websites

Redux Toolkit is used to manage shopping carts, product details, user login, and orders. It helps keep all the data updated across the application.

Social Media Apps

It is useful for handling user profiles, posts, likes, comments, and notifications, so every change is reflected quickly.

Food Delivery Apps

Food delivery applications use Redux Toolkit to manage menus, carts, order status, and delivery tracking.

Learning Platforms

It helps manage student information, courses, quizzes, and learning progress in one place.

My Understanding

After learning Redux Toolkit, I realized that it is very useful when multiple parts of an application need the same data. Instead of passing data from one component to another, Redux Toolkit keeps everything in one place, making the application easier to build and maintain.

Challenges I Faced

At the beginning, I found it difficult to understand concepts like Store, State, and Reducers. I was also confused about how data flows between components. After practicing with simple examples, these concepts became much clearer.

How I Solved Them

- Learned one concept at a time.
- Practiced with simple examples.
- Revised the basics whenever I got confused.

My Learning

These challenges helped me understand Redux Toolkit better and improved my confidence in using it for React projects.

Additional Things I Explored

While learning Redux Toolkit, I also explored a few related topics:

- Difference between Traditional Redux and Redux Toolkit.
- How React Redux connects components with the Store.
- Using Redux DevTools for debugging.
- Organizing projects with a proper folder structure.

My Learning

Exploring these topics helped me understand Redux Toolkit better and gave me more confidence to use it in React projects.

Conclusions

Through this mini hackathon, I learned that:

- I understood the basics of Redux Toolkit and state management.
- I learned the purpose of Store, State, Actions, Reducers, and Slices.
- I explored how data flows in a Redux Toolkit application.
- I understood the importance of a proper folder structure.
- I learned how Redux Toolkit reduces boilerplate code and makes Redux Toolkit makes React projects easier to manage.
- This documentation improved my confidence in using Redux Toolkit in future React projects.

References

- Official Redux Toolkit Documentation – <https://redux-toolkit.js.org/>
- React Documentation – <https://react.dev/>
- React Redux Documentation – <https://react-redux.js.org/>
- Sheryians Coding School – Redux Toolkit Complete Course (YouTube) – <https://youtu.be/Q5TqsetwCoE?si=ITIWBNZd5pOF5GnX>
- Personal notes and practice completed during the Mini Hackathon.